SUST_Ragebait_Tree

Shahjalal University of Science and Technology

1 Templates and Scripts 1
- 1.1 sublime 1
- 1.2 vs code 1
- 1.3 Interactor 1
- 1.4 debug.h 1
- 1.5 Template - Nahdi 1
- 1.6 Template - Ibnul 1
- 1.7 Mod Template 2

2 Number Theory 2
- 2.1 Binomial Coeffecient 2
- 2.2 Sieve 2
- 2.3 CRT 2

3 Data Structure 2
- 3.1 Fenwick Tree 2
- 3.2 Segment Tree 2
- 3.3 DSU 3
- 3.4 Persistent Segment Tree . . . 3

4 Graph 3
- 4.1 Dijkstra 3
- 4.2 LCA 3
- 4.3 SCC 4
- 4.4 HLD 4
- 4.5 Centroid Decomposition . . . 4

5 String 4
- 5.1 Trie 4
- 5.2 Z Algorithm 4

6 Geometry 4
- 6.1 Common 4
- 6.2 Vector2d 4
- 6.3 Vector3d 5

7 Miscellaneous 5
- 7.1 Generationg Permutations . . 5
- 7.2 Iterating over Submasks . . . 5
- 7.3 Random 5
- 7.4 pbds 5

8 formulae 5
- 8.1 Formulae 5

1 Templates and Scripts

1.1 sublime

```
{
  "shell_cmd": "g++ -Wall -std=c++20 -O2
      -DLOCAL $file -o $file_base_name
     && bash -c 'ulimit -S -f 10000;
     timeout 5 ./$file_base_name < input
     .in > output.out 2> error.log'",
  "file_regex": "^(..[^:]*):([0-9]+)
     :?([0-9]+)?:? (.*)$",
  "working_dir": "${file_path}",
  "selector": "source.c++, source.c,
     source.cpp",
  "variants":
  [
    {
      "name": "sanitize",
      "shell_cmd": "g++ -Wall -std=c++20
      -fsanitize=address -fsanitize=
     undefined -DLOCAL $file -o
     $file_base_name && bash -c 'ulimit
     -S -f 10000; timeout 5 ./
     $file_base_name < input.in > output
     .out 2> error.log'"
    }
  ]
}
```

1.2 vs code

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "cp",
      "type": "shell",
      "command": "",
      "args": [
        "g++",
        "-Wall",
        "-std=c++20",
        "-O2",
        "-DLOCAL",
        "\"${file}\"",
        "-o",
        "\"${fileDirname}/${
    fileBasenameNoExtension}\"",
        "&&",
        "\"${fileDirname}/${
    fileBasenameNoExtension}\"",
        "<",
        "input.in",
        ">",
        "output.out",
        "2>",
        "error.log"
      ],
      "group": "build",
      "presentation": {
        "reveal": "silent"
      },
      "problemMatcher": {
        "owner": "cpp",
        "fileLocation": ["relative", "${
    workspaceRoot}"],
        "pattern": {
          "regexp": "^(.*):(\\d+):(\\d+)
    :\\s+(warning|error):\\s+(.*)$",
          "file": 1,
          "line": 2,
          "column": 3,
          "severity": 4,
          "message": 5
        }
      }
    }
  ]
}
```

1.3 Interactor

```
socat EXEC:"./interactor" EXEC:"./soln"
```

1.4 debug.h

```
#include <iostream>
using namespace std;
#define see(x) cerr << #x << ": " << x
     << nl
template <typename... Args>
void debug(Args... args) {
  ((cerr << " " << args), ...) << "\n";
}
```

1.5 Template - Nahdi

```
#include <bits/stdc++.h>
using namespace std;
using vi = vector<int>;
using vii = vector<vi>;
using vs = vector<string>;
using ii = pair<int, int>;
using vpi = vector<ii>;
using ll = long long;
using vll = vector<ll>;
using pll = pair<ll, ll>;
#define f first
#define s second
#define forn(i, n) for (int i = 0; i < n
     ; i++)
#define fore(i, a, b) for (int i = a; i
     <= b; i++)
#define for1n(i, n) for (int i = 1; i <=
     n; i++)
#define all(x) x.begin(), x.end()
#define Flag cout << "Reached here.\n"
#define FASTIO              \
  ios::sync_with_stdio(0); \
  cin.tie(0);              \
  cout.tie(0);
#define pb push_back
#define pbb pop_back
#define sz size
#define rsz resize
#define ins insert
#define MAX 100000000011
#define MIN -MAX
#define MOD 1000000007LL
#define clr(x, y) memset(x, y, sizeof(x)
     )
int dx[] = {0, 1, -1, 0};
int dy[] = {1, 0, 0, -1};
void init() {
#ifndef ONLINE_JUDGE
  freopen("output.txt", "w", stdout);
  freopen("input.txt", "r", stdin);
#endif
}
void solve() {}
int main() {
  // init();
  // FASTIO;
  int t;
  cin >> t;
  while (t--) solve();
}
```

1.6 Template - Ibnul

```
#include <bits/stdc++.h>
using namespace std;
// Template
// ====================================
// pbds
// #include <ext/pb_ds/assoc_container.
     hpp>
// #include <ext/pb_ds/tree_policy.hpp>
// using namespace __gnu_pbds;
// template<typename T, typename comp =
     less<T>>
// using ordered_set =  tree<T,
     null_type, comp, rb_tree_tag,
// tree_order_statistics_node_update>;
// Debugging
#ifdef LOCAL
#include "debug.h"
#else
#define debug(...)
#define see(x)
#endif
typedef long long ll;
typedef vector<int> VI;
typedef vector<long long> VLL;
typedef vector<bool> VB;
typedef vector<vector<int>> VVI;
typedef pair<int, int> PI;
typedef pair<ll, ll> PLL;
typedef vector<pair<int, int>> VPI;
#define pb push_back
#define ff first
#define ss second
#define mp make_pair
#define all(a) a.begin(), a.end()
#define revall(a) a.rbegin(), a.rend()
#define loop(i, s, e) for (int i = s; i
     < e; ++i)
#define inp(v) \
  for (auto& x : v) cin >> x
#define outp(v, st) \
  for (int i = st, n = v.size(); i < n;
     ++i) cout << v[i] << " \n"[i == n -
     1]
#define nl "\n"
#define yep cout << "YES\n"
#define nope cout << "NO\n"
#define INF (int)1e9
```

```
#define INFL (ll)1e18
#define EPS 1e-9
#define MOD 998244353
// #define MOD 1000000007
#define MAXN 2002
// ===================================
void solve(int tc) {}
int main() {
  ios_base::sync_with_stdio(false);
  cin.tie(NULL);
  int t = 1;
  // cin >> t;
  for (int i = 1; i <= t; ++i) solve(i);
#ifdef LOCAL
  cerr << "Execution time: " << 1000.f *
      clock() / CLOCKS_PER_SEC << " ms."
       << nl;
#endif
  return 0;
}
```

1.7 Mod Template

```
template <unsigned P>
struct modint {
  unsigned v;
  modint() = default;
  template <class T>
  modint(T x) {
    x %= (signed)P, v = x < 0 ? x + P :
     x;
  }
  modint operator+() const { return *
     this; }
  modint operator-() const { return
     modint(0) - *this; }
  static modint power(modint a, int b) {
    if (b < 0) return power(a, -b).inv()
     ;
    modint res = 1;
    while (b) {
      if (b & 1) res *= a;
      a *= a;
      b >>= 1;
    }
    return res;
  }
  modint inv() const { return assert(v),
      power(*this, P - 2); }
  modint& operator+=(const modint& q) {
    (v += q.v) < P || (v -= P);
    return *this;
  }
  modint& operator-=(const modint& q) {
    (v -= q.v) < P || (v += P);
    return *this;
  }
  modint& operator*=(const modint& q) {
    v = 1ull * v * q.v % P;
    return *this;
  }
  modint& operator/=(const modint& q) {
    v = 1ull * v * q.inv().v % P;
    return *this;
  }
  friend modint operator+(modint p,
     const modint& q) { return p += q; }
  friend modint operator-(modint p,
     const modint& q) { return p -= q; }
  friend modint operator*(modint p,
     const modint& q) { return p *= q; }
  friend modint operator/(modint p,
     const modint& q) { return p /= q; }
  friend istream& operator>>(istream& is
     , modint& a) {
    int x;
    is >> x;
    x %= (signed)P;
    a.v = x < 0 ? x + P : x;
    return is;
  }
  friend ostream& operator<<(ostream& os
     , const modint& a) {
    os << a.v;
    return os;
  }
};
typedef modint<998244353> mint;
typedef modint<1000000007> mint2;
```

2 Number Theory

2.1 Binomial Coeffecient

```
ll fact[MAXN];
void calc_fact(ll mod = MOD) {
  fact[0] = 1;
  for (int i = 1; i < MAXN; ++i) {
    fact[i] = (fact[i - 1] * i) % mod;
  }
}
ll C(int n, int k, ll mod = MOD) {
  if (k > n) return 0;
  return (1LL * fact[n] * inv((1LL *
     fact[k] * fact[n - k]) % mod)) %
     mod;
}
```

2.2 Sieve

```
VI primes;
bool isprime[MAXN];
void sieve() {
  isprime[1] = false;
  for (int i = 2; i < MAXN; ++i) isprime
     [i] = true;
  for (int i = 2; i < MAXN; ++i) {
    if (!isprime[i]) continue;
    primes.pb(i);
    if (1LL * i * i >= MAXN) continue;
    for (int j = i * i; j < MAXN; j += i
     ) isprime[j] = false;
  }
}
```

2.3 CRT

```
int extended_gcd(ll a, ll b, ll& x, ll&
     y) {
  if (!b) {
    x = 1, y = 0;
    return a;
  }
  ll g = extended_gcd(b, a % b, x, y);
  swap(x, y);
  y -= x * (a / b);
  return g;
}
ll modinv(ll a, ll m) {
  ll x, y;
  extended_gcd(a, m, x, y);
  x %= m;
  return x < 0 ? x + m : x;
}
ll crt(vector<pair<ll, ll>>& mods) {
  ll mod = 1;
  for (auto& p : mods) mod *= p.ss;
  ll ans = 0;
  for (auto [x, p] : mods) {
    ll m = mod / p;
    m = (m * modinv(m, p)) % mod;
    m = (m * x) % mod;
    ans = (ans + m) % mod;
  }
  return ans;
}
```

3 Data Structure

3.1 Fenwick Tree

```
struct fenwick {
  int n;
  vector<ll> tree;
  fenwick(int s) {
    n = s;
    tree.assign(n + 1, 0);
  }
  ll get(int i) {
    ll res = 0;
    for (; i > 0; i -= i & -i) res +=
     tree[i];
    return res;
  }
  void add(int i, ll x) {
    for (; i <= n; i += i & -i) tree[i]
     += x;
  }
  void add(int l, int r, ll x) {
    add(l, x);
    if (r < n) add(r + 1, -x);
  }
  ll get(int l, int r) { return get(r) -
     get(l - 1); }
};
```

3.2 Segment Tree

```
template <typename T>
struct segtree {
#define left(u) 2 * u + 1
#define right(u) 2 * u + 2
  int n;
  T init_val;
  vector<T> tree;
  function<T(T, T)> f;
  bool is_lazy = false;
  T init_lazy;
  vector<T> lazy;
  function<void(T&, T, int, int)>
     apply_lazy;
  function<void(T&, T)> merge_lazy;
  segtree() {}  // Dummy constructor for
     global declaration
  segtree(int sz, function<T(T, T)> f, T
     val = 0) {
    n = sz;
    this->f = f;
    init_val = val;
    tree.assign(1 << (__lg(n - 1) + 2),
     init_val);
  }
  // apply(tree[u], lazy[u], l, r)
  // merge(lazy[left(u)], lazy[u])
  void make_lazy(function<void(T&, T,
     int, int)> apply,
                function<void(T&, T)>
     merge, T init = 0) {
    is_lazy = true;
    init_lazy = init;
    apply_lazy = apply;
    merge_lazy = merge;
    lazy.assign(tree.size(), init_lazy);
  }
  void build(vector<T>& a) { _build(0,
     0, n - 1, a); }
  void update(int i, T x) { _update(0,
     0, n - 1, i, x); }
  void update(int l, int r, T x) {
     _update_range(0, 0, n - 1, l, r, x)
     ; }
  T get(int l, int r) { return _get(0,
     0, n - 1, l, r); }
  void _build(int u, int l, int r,
     vector<T>& a) {
    if (l == r) {
      tree[u] = a[l];
      return;
    }
    int mid = (l + r) / 2;
    _build(left(u), l, mid, a);
    _build(right(u), mid + 1, r, a);
    tree[u] = f(tree[left(u)], tree[
     right(u)]);
  }
  void _update(int u, int l, int r, int
     i, T x) {
    if (l == r) {
      tree[u] = x;
      return;
    }
    int mid = (l + r) / 2;
    if (i <= mid)
      _update(left(u), l, mid, i, x);
    else
      _update(right(u), mid + 1, r, i, x
     );
    tree[u] = f(tree[left(u)], tree[
     right(u)]);
  }
  void propagate(int u, int l, int r) {
    if (lazy[u] == init_lazy || l == r)
     return;
```

```
    int m = (l + r) / 2;
    apply_lazy(tree[left(u)], lazy[u], l
     , m);
    merge_lazy(lazy[left(u)], lazy[u]);
    apply_lazy(tree[right(u)], lazy[u],
     m + 1, r);
    merge_lazy(lazy[right(u)], lazy[u]);
    lazy[u] = init_lazy;
  }
  void _update_range(int u, int tl, int
     tr, int l, int r, T x) {
    if (l > tr || r < tl) return;
    if (tl >= l && tr <= r) {
      apply_lazy(tree[u], x, tl, tr);
      merge_lazy(lazy[u], x);
      return;
    }
    propagate(u, tl, tr);
    int mid = (tl + tr) / 2;
    _update_range(left(u), tl, mid, l, r
     , x);
    _update_range(right(u), mid + 1, tr,
      l, r, x);
    tree[u] = f(tree[left(u)], tree[
     right(u)]);
  }
  T _get(int u, int tl, int tr, int l,
     int r) {
    if (l > tr || r < tl) return
     init_val;
    if (l <= tl && r >= tr) return tree[
     u];
    if (is_lazy) propagate(u, tl, tr);
    int mid = (tl + tr) / 2;
    return f(_get(left(u), tl, mid, l, r
     ), _get(right(u), mid + 1, tr, l, r
     ));
  }
#undef left
#undef right
};
```

3.3 DSU

```
struct dsu {
  vector<int> parent, size;
  dsu(int n) {
    parent.resize(n + 1);
    size.resize(n + 1);
    for (int i = 1; i <= n; ++i) {
      parent[i] = i;
      size[i] = 1;
    }
  }
  int find(int u) {
    if (parent[u] == u) return u;
    return parent[u] = find(parent[u]);
  }
  void merge(int u, int v) {
    u = find(u);
    v = find(v);
    if (u != v) {
      if (size[u] < size[v]) swap(u, v);
      parent[v] = u;
      size[u] += size[v];
    }
  }
};
```

3.4 Persistent Segment Tree

```
template <typename T>
struct node {
  T val;
  int l, r;
};
template <typename T>
struct persistent_segtree {
  int n;
  T init_val;
  vector<node<T>> nodes;
  vector<int> version;
  function<T(T, T)> f;
  persistent_segtree() {}
  persistent_segtree(int sz, function<T(
     T, T)> f, T val = 0) {
    n = sz;
    this->f = f;
    init_val = val;
    build(vector<T>(n, init_val));
  }
  persistent_segtree(vector<T>& a,
     function<T(T, T)> f, T val = 0) {
    n = a.size();
    this->f = f;
    init_val = val;
    build(a);
  }
  int build(vector<T> a) {
    int root = _build(0, n - 1, a);
    version.pb(root);
    return version.size() - 1;
  }
  int update(int i, T x, int v = -1) {
    if (v == -1) v = version.size() - 1;
    int root = _update(version[v], 0, n
     - 1, i, x);
    version.pb(root);
    return version.size() - 1;
  }
  T get(int l, int r, int v) { return
     _get(version[v], 0, n - 1, l, r); }
  // version is 1 indexed (0 is the
     initial before updates)
  T get_kth(int ver_l, int ver_r, int k)
     {
    return _get_kth(version[ver_l - 1],
     version[ver_r], 0, n - 1, k);
  }
  int _build(int l, int r, vector<T>& a)
     {
    int root = nodes.size();
    nodes.pb({});
    if (l == r) {
      nodes[root].val = a[l];
      return root;
    }
    int mid = (l + r) / 2;
    int new_child = _build(l, mid, a);
    nodes[root].l = new_child;
    new_child = _build(mid + 1, r, a);
    nodes[root].r = new_child;
    nodes[root].val = f(nodes[nodes[root
     ].l].val, nodes[nodes[root].r].val)
     ;
    return root;
  }
  int _update(int u, int l, int r, int i
     , T x) {
    int root = nodes.size();
    node<T> old = nodes[u];
    nodes.pb(old);
    if (l == r) {
      nodes[root].val = x;
      return root;
    }
    int mid = (l + r) / 2;
    if (i <= mid) {
      int newl = _update(nodes[u].l, l,
     mid, i, x);
      nodes[root].l = newl;
    } else {
      int newr = _update(nodes[u].r, mid
      + 1, r, i, x);
      nodes[root].r = newr;
    }
    nodes[root].val = f(nodes[nodes[root
     ].l].val, nodes[nodes[root].r].val)
     ;
    return root;
  }
  T _get(int u, int tl, int tr, int l,
     int r) {
    if (l > tr || r < tl) return
     init_val;
    if (l <= tl && r >= tr) return nodes
     [u].val;
    int mid = (tl + tr) / 2;
    return f(_get(nodes[u].l, tl, mid, l
     , r),
             _get(nodes[u].r, mid + 1,
     tr, l, r));
  }
  int _get_kth(int u, int v, int l, int
     r, int k) {
    if (l == r) return l;
    int mid = (l + r) / 2;
    int kl = nodes[nodes[v].l].val -
     nodes[nodes[u].l].val;
    if (kl >= k)
      return _get_kth(nodes[u].l, nodes[
     v].l, l, mid, k);
    else
      return _get_kth(nodes[u].r, nodes[
     v].r, mid + 1, r, k - kl);
  }
};
```

4 Graph

4.1 Dijkstra

```
struct Node {
  int u;
  ll dist;
  bool operator<(const Node& v) const {
     return dist > v.dist; }
};
vector<ll> dijkstra(int start, vector<
     VPI>& adj) {
  vector<ll> dis(adj.size(), INFL);
  priority_queue<Node> q;
  dis[start] = 0;
  q.push({start, 0});
  while (!q.empty()) {
    Node node = q.top();
    int u = node.u;
    q.pop();
    if (dis[u] < node.dist) continue;
    for (auto e : adj[u]) {
      if (node.dist + e.ss < dis[e.ff])
     {
        dis[e.ff] = node.dist + e.ss;
        q.push({e.ff, dis[e.ff]});
      }
    }
  }
  return dis;
}
```

4.2 LCA

```
struct LCA {
  int step = 0;
  VI vis_start, vis_end;
  VVI st;
  int k;
  VVI& tree;
  LCA(VVI& tree) : tree(tree) { init(
     tree.size() - 1); }
  void build_st(int u, int p) {
    st[u][0] = p;
    for (int i = 1; i <= k; ++i) st[u][i
     ] = st[st[u][i - 1]][i - 1];
  }
  void dfs(int u, int p) {
    vis_start[u] = step++;
    build_st(u, p);
    for (auto v : tree[u]) {
      if (v != p) dfs(v, u);
    }
    vis_end[u] = step++;
  }
  bool is_ancestor(int a, int b) {
    return vis_start[a] < vis_start[b]
     && vis_end[a] > vis_end[b];
  }
  int get(int a, int b) {
    if (a == b) return a;
    if (is_ancestor(a, b)) return a;
    if (is_ancestor(b, a)) return b;
    for (int i = k; i >= 0; --i) {
      if (!is_ancestor(st[a][i], b)) a =
      st[a][i];
    }
    return st[a][0];
  }
  void init(int n) {
    vis_start.assign(n + 1, 0);
    vis_end.assign(n + 1, 0);
    k = __lg(2 * n - 1);
    st.assign(n + 1, VI(k + 1));
    dfs(1, 1);
  }
  // Extra
```

```
  int get_ancestor(int u, int n) {
    for (int i = 0; n > 0; ++i) {
      if (n & (1 << i)) {
        u = st[u][i];
        n -= (1 << i);
      }
    }
    return u;
  }
};
```

4.3 SCC

```
VVI scc;
VI stk;
int disc[MAXN], low[MAXN];
bool instk[MAXN];
int cur_time;
void dfs_scc(int u, VVI& g) {
  disc[u] = low[u] = ++cur_time;
  instk[u] = true;
  stk.pb(u);
  for (int v : g[u]) {
    if (!disc[v]) {
      dfs_scc(v, g);
      low[u] = min(low[u], low[v]);
    } else if (instk[v])
      low[u] = min(low[u], disc[v]);
  }
  if (low[u] < disc[u]) return;
  scc.pb(VI());
  bool rem = true;
  while (rem) {
    int v = stk.back();
    stk.pop_back();
    instk[v] = false;
    scc.back().pb(v);
    rem = v != u;
  }
}
void tarjan(VVI& g) {
  cur_time = 0;
  int n = g.size();
  for (int i = 0; i < n; ++i) disc[i] =
     0;
  for (int u = 1; u < n; ++u)
    if (!disc[u]) dfs_scc(u, g);
}
```

4.4 HLD

```
VI heavy, chain, label, par, sz, dep;
int timer;
void dfsz(int u, int p, VVI& tree) {
  sz[u] = 1;
  par[u] = p;
  dep[u] = dep[p] + 1;
  int mx = -1;
  for (int v : tree[u]) {
    if (v == p) continue;
    dfsz(v, u, tree);
    sz[u] += sz[v];
    if (sz[v] > mx) mx = sz[v], heavy[u]
      = v;
  }
}
void dfshld(int u, int p, VVI& tree, int
      head) {
  label[u] = timer++;
  chain[u] = head;
  if (heavy[u] != -1) dfshld(heavy[u], u
     , tree, head);
  for (int v : tree[u]) {
    if (v == p || v == heavy[u])
     continue;
    dfshld(v, u, tree, v);
  }
}
void inithld(int n, VVI& tree) {
  heavy = VI(n + 1, -1);
  chain = VI(n + 1);
  label = VI(n + 1);
  sz = VI(n + 1);
  par = VI(n + 1);
  dep = VI(n + 1);
  timer = 1;
  dfsz(1, 0, tree);
  dfshld(1, 0, tree, 1);
}
ll query(int u, int v, fenwick& bit) {
  ll ret = 0;  // init value
  for (; chain[u] != chain[v]; v = par[
     chain[v]]) {
    if (dep[chain[u]] > dep[chain[v]])
     swap(u, v);
    // ret ?= bit.rsq(label[chain[v]],
     label[v]);
  }
  if (dep[u] > dep[v]) swap(u, v);
  // lca = u
  // ret ?= bit.rsq(label[u], label[v]);
  return ret;
}
```

4.5 Centroid Decomposition

```
struct CD {
  vector<set<int>> tree;
  vector<int> par, sz;
  CD(vector<vector<int>>& tree) {
    for (auto& v : tree) {
      this->tree.emplace_back(v.begin(),
      v.end());
    }
    par.resize(tree.size());
    sz.resize(tree.size());
    build(1, 0);
  }
  void build(int u, int p) {
    dfs(u, p);
    int centroid = dfs(u, p, sz[u]);
    par[centroid] = p;
    vector<int> adj(tree[centroid].begin
     (), tree[centroid].end());
    for (int v : adj) {
      tree[v].erase(centroid);
      tree[centroid].erase(v);
      build(v, centroid);
    }
  }
  void dfs(int u, int p) {
    sz[u] = 1;
    for (int v : tree[u]) {
      if (v == p) continue;
      dfs(v, u);
      sz[u] += sz[v];
    }
  }
  int dfs(int u, int p, int n) {
    for (int v : tree[u]) {
      if (v == p) continue;
      if (sz[v] > n / 2) return dfs(v, u
     , n);
    }
    return u;
  }
  int operator[](int i) { return par[i];
      }
};
```

5 String

5.1 Trie

```
struct trienode {
  bool endmark;
  vector<int> child;
  trienode(int sz) {
    endmark = false;
    child.resize(sz, 0);
  }
};
struct trie {
  int sz;
  char fst;
  vector<trienode> nodes;
  trie(int alpha_sz, char alpha_start) {
    sz = alpha_sz;
    fst = alpha_start;
    // root at idx 0
    nodes.pb(trienode(sz));
  }
  void insert(string& s) {
    int cur = 0;
    for (char c : s) {
      if (!nodes[cur].child[c - fst]) {
        nodes[cur].child[c - fst] =
     nodes.size();
        nodes.pb(trienode(sz));
      }
      cur = nodes[cur].child[c - fst];
    }
    nodes[cur].endmark = true;
  }
  bool search(string& s) {
    int cur = 0;
    for (char c : s) {
      if (!nodes[cur].child[c - fst])
     return false;
      cur = nodes[cur].child[c - fst];
    }
    return nodes[cur].endmark;
  }
  bool erase(string& s) {
    int cur = 0;
    for (char c : s) {
      if (!nodes[cur].child[c - fst])
     return false;
      cur = nodes[cur].child[c - fst];
    }
    if (!nodes[cur].endmark) return
     false;
    nodes[cur].endmark = false;
    return true;
  }
};
```

5.2 Z Algorithm

```
VI Z(string& s) {
  int n = s.size();
  VI z(n);
  int l = 0, r = 0;
  for (int i = 0; i < n; ++i) {
    if (i <= r) z[i] = min(z[i - l], r -
      i + 1);
    while (i + z[i] < n && s[z[i]] == s[
     i + z[i]]) l = i, r = i + z[i]++;
  }
  return z;
}
```

6 Geometry

6.1 Common

```
#define EPS 1e-12
#define pi 3.14159265358979311599796 34\
  6854418516159057617187 5L
bool feq(const long double& a, const
     long double& b) {
  long double del = fabsl(a - b);
  if (del <= EPS) return true;
  return del <= EPS * max(fabsl(a),
     fabsl(b));
}
```

6.2 Vector2d

```
struct vector2d {
  long double x, y;
  bool operator==(const vector2d& p)
     const {
    return feq(x, p.x) && feq(y, p.y);
  }
  vector2d operator+(const vector2d& p)
     const {
    return {x + p.x, y + p.y};
  }
  vector2d operator-(const vector2d& p)
     const {
    return {x - p.x, y - p.y};
  }
  vector2d operator*(long double scalar)
     const {
    return {scalar * x, scalar * y};
  }
  friend vector2d operator*(long double
     s, const vector2d& v) {
    return {v.x * s, v.y * s};
  }
```

```
  vector2d operator/(long double scalar)
      const {
    return {x / scalar, y / scalar};
  }
  long double norm2() const {
    return x * x + y * y;
  }
  long double magnitude() const {
    return sqrtl(norm2());
  }
  vector2d unit() const {
    long double m = magnitude();
    if (feq(m, 0)) return {0, 0};
    return {x / m, y / m};
  }
  vector2d normal() const {
    vector2d v{y, -x};
    return v.unit();
  }
};
long double dist(const vector2d& p1,
     const vector2d& p2) {
  return (p1 - p2).magnitude();
}
long double dot(const vector2d& v1,
     const vector2d& v2) {
  return v1.x * v2.x + v1.y * v2.y;
}
long double angle(const vector2d& v1,
     const vector2d& v2) {
  long double m = v1.magnitude() * v2.
     magnitude();
  if (feq(m, 0)) return 0;
  long double v = dot(v1, v2) / m;
  return acosl(min(1.0L, max(-1.0L, v)))
     ;
}
long double cross(const vector2d& v1,
     const vector2d& v2) {
  return v1.x * v2.y - v1.y * v2.x;
}
```

6.3 Vector3d

```
struct vector3d {
  long double x, y, z;
  bool operator==(const vector3d& p)
     const {
    return feq(x, p.x) && feq(y, p.y) &&
      feq(z, p.z);
  }
  vector3d operator+(const vector3d& p)
     const {
    return {x + p.x, y + p.y, z + p.z};
  }
  vector3d operator-(const vector3d& p)
     const {
    return {x - p.x, y - p.y, z - p.z};
  }
  vector3d operator*(long double scalar)
      const {
    return {x * scalar, y * scalar, z *
     scalar};
  }
  friend vector3d operator*(long double
     s, const vector3d& v) {
    return {v.x * s, v.y * s, v.z * s};
  }
  vector3d operator/(long double scalar)
      const {
    return {x / scalar, y / scalar, z /
     scalar};
  }
  long double norm2() const {
    return x * x + y * y + z * z;
  }
  long double magnitude() const {
    return sqrtl(norm2());
  }
  vector3d unit() const {
    long double m = magnitude();
    if (feq(m, 0)) return {0, 0, 0};
    return *this / m;
  }
};
long double dist(const vector3d& p1,
     const vector3d& p2) {
  return (p1 - p2).magnitude();
}
long double dot(const vector3d& v1,
     const vector3d& v2) {
  return v1.x * v2.x + v1.y * v2.y + v1.
     z * v2.z;
}
long double angle(const vector3d& v1,
     const vector3d& v2) {
  long double m = v1.magnitude() * v2.
     magnitude();
  if (feq(m, 0)) return 0;
  long double v = dot(v1, v2) / m;
  return acosl(min(1.0L, max(-1.0L, v)))
     ;
}
vector3d cross(const vector3d& a, const
     vector3d& b) {
  return {
    a.y * b.z - a.z * b.y,
    a.z * b.x - a.x * b.z,
    a.x * b.y - a.y * b.x
  };
}
```

7 Miscellaneous

7.1 Generationg Permutations

```
VI a;
//...
sort(all(a));
do {
  // Process permutation...
} while (next_permutation(all(a)));
```

7.2 Iterating over Submasks

```
int n = 17;
for (int mask = 0; mask < (1 << n); ++
     mask) {
  for (int sub = mask; sub; sub = (sub -
      1) & mask) {
    // Process submask...
  }
}
```

7.3 Random

```
mt19937 rng(chrono::steady_clock::now().
     time_since_epoch().count());
int randint(int l, int r) {
  uniform_int_distribution<int> dist(l,
     r);
  return dist(rng);
}
```

7.4 pbds

```
// ordered set
ordered_set<int> st;
// ordered multiset
ordered_set<int, less_equal<int>> multi;
// insert element
st.insert(x);
// get index
st.order_of_key(x);
// access by index
st.find_by_order(idx);
```

8 formulae

8.1 Formulae

8.1.1 Combinatorics

1. $\sum_{0 \le k \le n} \binom{n-k}{k} = Fib_{n+1}$
2. $\binom{n}{k} = \binom{n}{n-k}$
3. $\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1}$
4. $k\binom{n}{k} = n\binom{n-1}{k-1}$
5. $\binom{n}{k} = \frac{n}{k}\binom{n-1}{k-1}$
6. $\sum_{i=0}^{n} \binom{n}{i} = 2^n$
7. $\sum_{i \ge 0} \binom{n}{2i} = 2^{n-1}$
8. $\sum_{i \ge 0} \binom{n}{2i+1} = 2^{n-1}$
9. $\sum_{i=0}^{k} (-1)^i \binom{n}{i} = (-1)^k \binom{n-1}{k}$
10. $\sum_{i=0}^{k} \binom{n+i}{i} = \sum_{i=0}^{k} \binom{n+i}{n} = \binom{n+k+1}{k}$
11. $1\binom{n}{1} + 2\binom{n}{2} + 3\binom{n}{3} + \cdots + n\binom{n}{n} = n2^{n-1}$
12. $1^2\binom{n}{1} + 2^2\binom{n}{2} + 3^2\binom{n}{3} + \cdots + n^2\binom{n}{n} = (n+n^2)2^{n-2}$
13. **Vandermonde's Identify:** $\sum_{k=0}^{r} \binom{m}{k}\binom{n}{r-k} = \binom{m+n}{r}$
14. **Hockey-Stick Identify:** $n, r \in N, n > r, \sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1}$
15. $\sum_{i=0}^{k} \binom{k}{i}^2 = \binom{2k}{k}$
16. $\sum_{k=0}^{n} \binom{n}{k}\binom{n}{n-k} = \binom{2n}{n}$
17. $\sum_{k=q}^{n} \binom{n}{k}\binom{k}{q} = 2^{n-q}\binom{n}{q}$
18. $\sum_{i=0}^{n} k^i \binom{n}{i} = (k+1)^n$
19. $\sum_{i=0}^{n} \binom{2n}{i} = 2^{2n-1} + \frac{1}{2}\binom{2n}{n}$
20. $\sum_{i=1}^{n} \binom{n}{i}\binom{n-1}{i-1} = \binom{2n-1}{n-1}$
21. $\sum_{i=0}^{n} \binom{2n}{i}^2 = \frac{1}{2}\left(\binom{4n}{2n} + \binom{2n}{n}^2\right)$
22. **Highest Power of 2 that divides ${}^{2n}C_n$:** Let x be the number of 1s in the binary representation. Then the number of odd terms will be 2^x. Let it form a sequence. The n-th value in the sequence (starting from $n = 0$) gives the highest power of 2 that divides ${}^{2n}C_n$.
23. **Pascal Triangle**
    (a) In a row p where p is a prime number, all the terms in that row except the 1s are multiples of p.
    (b) Parity: To count odd terms in row n, convert n to binary. Let x be the number of 1s in the binary representation. Then the number of odd terms will be 2^x.
    (c) Every entry in row $2^n - 1, n \ge 0$, is odd.
24. An integer $n \ge 2$ is prime if and only if all the intermediate binomial coefficients $\binom{n}{1}, \binom{n}{2}, \ldots, \binom{n}{n-1}$ are divisible by n.
25. **Kummer's Theorem:** For given integers $n \ge m \ge 0$ and a prime number p, the largest power of p dividing $\binom{n}{m}$ is equal to the number of carries when m is added to n-m in base p. For implementation take inspiration from lucas theorem.
26. Number of different binary sequences of length n such that no two 0's are adjacent$=Fib_{n+1}$
27. **Combination with repetition:** Let's say we choose k elements from an n-element set, the order doesn't matter and each element can be chosen more than once. In that case, the number of different combinations is: $\binom{n+k-1}{k}$

28. Number of ways to divide n persons in $\frac{n}{k}$ equal groups i.e. each having size k is
$$\frac{n!}{k!^{\frac{n}{k}}\left(\frac{n}{k}\right)!} = \prod_{n\geq k}^{n-=k}\binom{n-1}{k-1}$$

29. The number non-negative solution of the equation: $x_1 + x_2 + x_3 + \cdots + x_k = n$ is$\binom{n+k-1}{n}$

30. Number of ways to choose n ids from 1 to b such that every id has distance at least k $= \left(\frac{b-(n-1)(k-1)}{n}\right)$

31. $\sum_{i=1,3,5,\ldots}^{i\leq n}\binom{n}{i}a^{n-i}b^i = \frac{1}{2}((a+b)^n - (a-b)^n)$

32. $\sum_{i=0}^{n}\frac{\binom{k}{i}}{\binom{n}{i}} = \frac{\binom{n+1}{n-k+1}}{\binom{n}{k}}$

33. **Derangement:** a permutation of the elements of a set, such that no element appears in its original position. Let $d(n)$ be the number of derangements of the identity permutation fo size n.
$d(n) = (n-1)\big(d(n-1) + d(n-2)\big)$, where $d(0) = 1,\ d(1) = 0$.

34. **Involutions:** permutations such that $p^2 =$ identity permutation. $a_0 = a_1 = 1$ and $a_n = a_{n-1} + (n-1)a_{n-2}$ for $n > 1$.

35. Let $T(n,k)$ be the number of permutations of size n for which all cycles have length $\leq k$.
$$T(n,k) = \begin{cases} n! & ; n \leq k \\ n\cdot T(n-1,k) & \\ -F(n-1,k) & \\ \cdot T(n-k-1,k) & ; n > k \end{cases}$$
Here $F(n,k) = n\cdot(n-1)\cdot\cdots\cdot(n-k+1)$

36. **Lucas Theorem**
    (a) If p is prime, then $\left(\frac{p^a}{k}\right) \equiv 0(\mod p)$
    (b) For non-negative integers m and n and a prime p, the following congruence relation holds: $\left(\frac{m}{n}\right) \equiv \prod_{i=0}^{k}\left(\frac{m_i}{n_i}\right)(\mod p)$, where, $m = m_k p^k + m_{k-1}p^{k-1} + \cdots + m_1 p + m_0$, and $n = n_k p^k + n_{k-1}p^{k-1} + \cdots + n_1 p + n_0$ are the base p expansions of m and n respectively. This uses the convention that $\left(\frac{m}{n}\right) = 0$,when $m < n$.

37. $\sum_{i=0}^{n}\binom{n}{i}\cdot i^k$
$$= \sum_{i=0}^{n}\binom{n}{i}\cdot\sum_{j=0}^{k}\left\{{k \atop j}\right\}\cdot i^{\underline{j}}$$
$$= \sum_{i=0}^{n}\binom{n}{i}\cdot\sum_{j=0}^{k}\left\{{k \atop j}\right\}\cdot j!\binom{n}{i}$$
$$= \sum_{i=0}^{n}\frac{n!}{(n-i)!}\cdot\sum_{j=0}^{k}\left\{{k \atop j}\right\}\cdot\frac{1}{(i-j)!}$$
$$= \sum_{i=0}^{n}\sum_{j=0}^{k}\frac{n!}{(n-i)!}\cdot\left\{{k \atop j}\right\}\cdot\frac{1}{(i-j)!}$$
$$= \text{n!}\sum_{i=0}^{n}\sum_{j=0}^{k}\left\{{k \atop j}\right\}\cdot\frac{1}{(n-i)!}\cdot\frac{1}{(i-j)!}$$
$$= \text{n!}\sum_{i=0}^{n}\sum_{j=0}^{k}\left\{{k \atop j}\right\}\cdot\binom{n-j}{n-i}\cdot\frac{1}{(n-j)!}$$
$$= \text{n!}\sum_{j=0}^{k}\left\{{k \atop j}\right\}\cdot\frac{1}{(n-j)!}\sum_{i=0}^{n}\cdot\binom{n-j}{n-i}$$
$$= \sum_{j=0}^{k}\left\{{k \atop j}\right\}\cdot n^{\underline{j}}\cdot 2^{n-j}$$
Here $n^{\underline{j}} = P(n,j) = \frac{n!}{(n-j)!}$ and $\left\{{k \atop j}\right\}$ is stirling number of the second kind.
So, instead of $O(n)$, now you can calculate the original equation in $O(k^2)$ or even in $O(k\log^2 n)$ using NTT.

38. $\sum_{i=0}^{n-1}\binom{i}{j}x^i = x^j(1-x)^{-j-1}$
$$\cdot\left(1 - x^n\sum_{i=0}^{j}\binom{n}{i}x^{j-i}(1-x)^i\right)$$

39. $x_0, x_1, x_2, x_3, \ldots, x_n$
$x_0 + x_1, x_1 + x_2, x_2 + x_3, \ldots x_n$
...
If we continuously do this n times then the polynomial of the first column of the n-th row will be
$$p(n) = \sum_{k=0}^{n}\binom{n}{k}\cdot x(k)$$

40. If $P(n) = \sum_{k=0}^{n}\binom{n}{k}\cdot Q(k)$, then,
$$Q(n) = \sum_{k=0}^{n}(-1)^{n-k}\binom{n}{k}\cdot P(k)$$

41. If $P(n) = \sum_{k=0}^{n}(-1)^k\binom{n}{k}\cdot Q(k)$, then,
$$Q(n) = \sum_{k=0}^{n}(-1)^k\binom{n}{k}\cdot P(k)$$

8.1.2 Catalan Numbers

42. $C_n = \frac{1}{n+1}\binom{2n}{n}$

43. $C_0 = 1, C_1 = 1$ and $C_n = \sum_{k=0}^{n-1}C_k C_{n-1-k}$

44. Number of correct bracket sequence consisting of n opening and n closing brackets.

45. The number of ways to completely parenthesize n+1 factors.

46. The number of triangulations of a convex polygon with n+2 sides (i.e. the number of partitions of polygon into disjoint triangles by using the diagonals).

47. The number of ways to connect the $2n$ points on a circle to form n disjoint i.e. non-intersecting chords.

48. The number of monotonic lattice paths from point $(0,0)$ to point (n,n) in a square lattice of size $n\times n$, which do not pass above the main diagonal (i.e. connecting $(0,0)$ to (n,n)).

49. The number of rooted full binary trees with n+1 leaves (vertices are not numbered). A rooted binary tree is full if every vertex has either two children or no children.

50. Number of permutations of $\{1,\ldots,n\}$ that avoid the pattern 123 (or any of the other patterns of length 3); that is, the number of permutations with no three-term increasing subsequence. For $n = 3$, these permutations are $132, 213, 231, 312$ and 321. For $n = 4$, they are $1432, 2143, 2413, 2431, 3142, 3214, 3241, 3412, 3421, 4132, 4213, 4231, 4312$ and 4321.

51. **Balanced Parentheses count with prefix:** The count of balanced parentheses sequences consisting of $n+k$ pairs of parentheses where the first k symbols are open brackets. Let the number be $C_n^{(k)}$, then
$$C_n^{(k)} = \frac{k+1}{n+k+1}\binom{2n+k}{n}$$

8.1.3 Narayana numbers

52. $N(n,k) = \frac{1}{n}\left(\frac{n}{k}\right)\left(\frac{n}{k-1}\right)$

53. The number of expressions containing n pairs of parentheses, which are correctly matched and which contain k distinct nestings. For instance, $N(4,2) = 6$ as with four pairs of parentheses six sequences can be created which each contain two times the sub-pattern '()'.

8.1.4 Stirling numbers of the first kind

54. The Stirling numbers of the first kind count permutations according to their number of cycles (counting fixed points as cycles of length one).

55. $S(n,k)$ counts the number of permutations of n elements with k disjoint cycles.

56. $S(n,k) = (n-1)\cdot S(n-1,k) + S(n-1,k-1)$, $where,\ S(0,0) = 1, S(n,0) = S(0,n) = 0$

57. $\sum_{k=0}^{n}S(n,k) = n!$

58. The unsigned Stirling numbers may also be defined algebraically, as the coefficient of the rising factorial:
$$x^{\bar{n}} = x(x+1)\ldots(x+n-1) = \sum_{k=0}^{n}S(n,k)x^k$$

59. Lets $[n,k]$ be the stirling number of the first kind, then
$$\left[{n \atop n-k}\right] = \sum_{0\leq i_1<i_2<i_k<n} i_1 i_2\cdots_k.$$

8.1.5 Stirling numbers of the second kind

60. Stirling number of the second kind is the number of ways to partition a set of n objects into k non-empty subsets.

61. $S(n,k) = k\cdot S(n-1,k) + S(n-1,k-1)$, $where\ S(0,0) = 1, S(n,0) = S(0,n) = 0$

62. $S(n,2) = 2^{n-1} - 1$

63. $S(n,k)\cdot k! =$ number of ways to color n nodes using colors from 1 to k such that each color is used at least once.

64. An r-associated Stirling number of the second kind is the number of ways to partition a set of n objects into k subsets, with each subset containing at least r elements. It is denoted by $S_r(n,k)$ and obeys the recurrence relation. $S_r(n+1,k) = kS_r(n,k) + \binom{n}{r-1}S_r(n-r+1,k-1)$

65. Denote the n objects to partition by the integers $1,2,\ldots,n$. Define the reduced Stirling numbers of the second kind, denoted $S^d(n,k)$, to be the number of ways to partition the integers $1,2,\ldots,n$ into k nonempty subsets such that all elements in each subset have pairwise distance at least d. That is, for any integers i and j in a given subset, it is required that $|i-j| \geq d$. It has been shown that these numbers satisfy, $S^d(n,k) = S(n-d+1,k-d+1), n\geq k\geq d$

8.1.6 Bell number

66. Counts the number of partitions of a set.

67. $B_{n+1} = \sum_{k=0}^{n}\left(\frac{n}{k}\right)\cdot B_k$

68. $B_n = \sum_{k=0}^{n}S(n,k)$,where $S(n,k)$ is stirling number of second kind.

8.1.7 Math

69. $ab \mod ac = a(b \mod c)$

70. $\sum_{i=0}^{n}i\cdot i! = (n+1)! - 1.$

71. $a^k - b^k = (a-b)\cdot(a^{k-1}b^0 + a^{k-2}b^1 + \cdots + a^0 b^{k-1})$

72. $\min(a+b,c) = a + \min(b,c-a)$

73. $|a-b| + |b-c| + |c-a| = 2(\max(a,b,c) - \min(a,b,c))$

74. $a\cdot b \leq c \to a \leq \left\lfloor\frac{c}{b}\right\rfloor$ is correct

75. $a\cdot b < c \to a < \left\lfloor\frac{c}{b}\right\rfloor$ is incorrect

76. $a\cdot b \geq c \to a \geq \left\lfloor\frac{c}{b}\right\rfloor$ is correct

77. $a\cdot b > c \to a > \left\lfloor\frac{c}{b}\right\rfloor$ is correct

78. For positive integer n, and arbitrary real numbers m, x, $\left\lfloor\frac{\lfloor x/m\rfloor}{n}\right\rfloor = \left\lfloor\frac{x}{mn}\right\rfloor$ $\left\lceil\frac{\lceil x/m\rceil}{n}\right\rceil = \left\lceil\frac{x}{mn}\right\rceil$

79. Lagrange's identity:

$$\left(\sum_{k=1}^{n} a_k^2\right)\left(\sum_{k=1}^{n} b_k^2\right) - \left(\sum_{k=1}^{n} a_k b_k\right)^2$$
$$= \sum_{i=1}^{n-1}\sum_{j=i+1}^{n}(a_i b_j - a_j b_i)^2$$
$$= \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1, j\neq i}^{n}(a_i b_j - a_j b_i)^2$$

80. $\sum_{i=1}^{n} i a^i = \frac{a(na^{n+1} - (n+1)a^n + 1)}{(a-1)^2}$

81. **Vieta's formulas:** Any general polynomial of degree n

$p(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0$

(with the coefficients being real or complex numbers and $a_n \neq 0$) is known by the fundamental theorem of algebra to have n (not necessarily distinct) complex roots $r_1, r_2, \ldots, r_n$.

$$\begin{cases} r_1 + r_2 + \cdots + r_{n-1} + r_n = -\frac{a_{n-1}}{a_n} \\ (r_1 r_2 + r_1 r_3 + \cdots + r_1 r_n) \\ +(r_2 r_3 + r_2 r_4 + \cdots + r_2 r_n) + \ldots \\ +r_{n-1} r_n = \frac{a_{n-2}}{a_n} \\ \vdots \\ r_1 r_2 \ldots r_n = (-1)^n \frac{a_0}{a_n}. \end{cases}$$

Vieta's formulas can equivalently be written as

$$\sum_{1\le i_1<i_2<\cdots<i_k\le n}\left(\prod_{j=1}^{k} r_{i_j}\right) = (-1)^k \frac{a_{n-k}}{a_n},$$

82. We are given n numbers $a_1, a_2, \ldots, a_n$ and our task is to find a value x that minimizes the sum,

$$|a_1 - x| + |a_2 - x| + \cdots + |a_n - x|$$

optimal x = median of the array. if n is even x = [left median,right median] i.e. every number in this range will work. For minimizing

$$(a_1 - x)^2 + (a_2 - x)^2 + \cdots + (a_n - x)^2$$

optimal $x = \frac{(a_1 + a_2 + \cdots + a_n)}{n}$

83. Given an array a of n non-negative integers. The task is to find the sum of the product of elements of all the possible subsets. It is equal to the product of $(a_i + 1)$ for all a_i

84. **Pentagonal number theorem:** In mathematics, the pentagonal number theorem states that

$\prod_{n=1}^{\infty}(1 - x^n) = \sum_{k=-\infty}^{\infty}(-1)^k x^{\frac{k(3k-1)}{2}}$
$= 1 + \sum_{k=1}^{\infty}(-1)^k\left(x^{\frac{k(3k+1)}{2}} + x^{\frac{k(3k-1)}{2}}\right).$

In other words,

$$(1-x)(1-x^2)(1-x^3)\cdots$$
$$= 1 - x - x^2 + x^5 + x^7 - x^{12}$$
$$-x^{15} + x^{22} + x^{26} - \cdots.$$

The exponents $1, 2, 5, 7, 12, \cdots$ on the right hand side are given by the formula $g_k = \frac{k(3k-1)}{2}$ for $k = 1, -1, 2, -2, 3, \cdots$ and are called (generalized) pentagonal numbers. It is useful to find the partition number in $O(n\sqrt{n})$

8.1.8 Fibonacci Number

85. $F_0 = 0, F_1 = 1$ and $F_n = F_{n-1} + F_{n-2}$

86. $F_n = \sum_{k=0}^{\lfloor\frac{n-1}{2}\rfloor}\binom{n-k-1}{k}$

87. $F_n = \frac{1}{\sqrt{5}}(\frac{1+\sqrt{5}}{2})^n - \frac{1}{\sqrt{5}}(\frac{1-\sqrt{5}}{2})^n$

88. $\sum_{i=1}^{n} F_i = F_{n+2} - 1$

89. $\sum_{i=0}^{n-1} F_{2i+1} = F_{2n}$

90. $\sum_{i=1}^{n} F_{2i} = F_{2n+1} - 1$

91. $\sum_{i=1}^{n} F_i^2 = F_n F_{n+1}$

92. $F_m F_{n+1} - F_{m-1}F_n = (-1)^n F_{m-n}$ $F_{2n} = F_{n+1}^2 - F_{n-1}^2 = F_n(F_{n+1} + F_{n-1})$

93. $F_m F_n + F_{m-1}F_{n-1} = F_{m+n-1}$ $F_m F_{n+1} + F_{m-1}F_n = F_{m+n}$

94. A number is Fibonacci if and only if one or both of $(5\cdot n^2 + 4)$ or $(5\cdot n^2 - 4)$ is a perfect square

95. Every third number of the sequence is even and more generally, every k^{th} number of the sequence is a multiple of F_k

96. $gcd(F_m, F_n) = F_{gcd(m,n)}$

97. Any three consecutive Fibonacci numbers are pairwise coprime, which means that, for every n, $gcd(F_n, F_{n+1}) = gcd(F_n, F_{n+2}), gcd(F_{n+1}, F_{n+2}) = 1$

98. If the members of the Fibonacci sequence are taken $mod\ n$, the resulting sequence is periodic with period at most $6n$.

8.1.9 Pythagorean Triples

99. A Pythagorean triple consists of three positive integers a, b, and C, such that $a^2 + b^2 = c^2$. Such a triple is commonly written (a, b, c)

100. Euclid's formula is a fundamental formula for generating Pythagorean triples given an arbitrary pair of integers m and n with $m > n > 0$. The formula states that the integers

$$a = m^2 - n^2, b = 2mn, c = m^2 + n^2$$

form a Pythagorean triple. The triple generated by Euclid's formula is primitive if and only if m and n are coprime and not both odd. When both m and n are odd, then a, b, and c will be even, and the triple will not be primitive; however, dividing a, b, and c by 2 will yield a primitive triple when m and n are coprime and both odd.

101. The following will generate all Pythagorean triples uniquely:

$$a = k\,(m^2 - n^2),$$
$$b = k\,(2mn), c = k\,(m^2 + n^2).$$

where m, n, and k are positive integers with $m > n$, and with m and n coprime and not both odd.

102. **Theorem:** The number of Pythagorean triples a,b,n with $max a, b, n = n$ is given by

$$\frac{1}{2}\left(\prod_{p^\alpha || n}(2\alpha + 1) - 1\right)$$

where the product is over all prime divisors p of the form $4k + 1$. The notation $p^\alpha || n$ stands for the highest exponent α for which p^α divides n **Example:** For $n = 2\cdot 3^2\cdot 5^3\cdot 7^4\cdot 11^5\cdot 13^6$, the number of Pythagorean triples with hypotenuse n is $\frac{1}{2}(7.13 - 1) = 45$. To obtain a formula for the number of Pythagorean triples with hypotenuse less than a specific positive integer N, we may add the numbers corresponding to each $n < N$ given by the Theorem. There is no simple way to compute this as a function of N.

8.1.10 Sum of Squares Function

103. The function is defined as

$$r_k(n) = \left|\begin{matrix}(a_1, a_2, \ldots, a_k) \in \mathbf{Z}^k : \\ n = a_1^2 + a_2^2 + \cdots + a_k^2\end{matrix}\right|$$

104. The number of ways to write a natural number as sum of two squares is given by $r_2(n)$. It is given explicitly by $r_2(n) = 4(d_1(n) - d_3(n))$ where d1(n) is the number of divisors of n which are congruent with 1 modulo 4 and d3(n) is the number of divisors of n which are congruent with 3 modulo 4. The prime factorization $n = 2^g p_1^{f_1} p_2^{f_2} \ldots q_1^{h_1} q_2^{h_2} \ldots$, where p_i are the prime factors of the form $p_i \equiv 1 \pmod 4$, and q_i are the prime factors of the form $q_i \equiv 3 \pmod 4$ gives another formula $r_2(n) = 4(f_1 + 1)(f_2 + 1)\ldots$, if all exponents h_1,h_2,... are even. If one or more h_i are odd, then $r_2(n) = 0$.

105. The number of ways to represent n as the sum of four squares is eight times the sum of all its divisors which are not divisible by 4, i.e. $r_4(n) = 8\sum_{d|n;4\nmid d} d$ $r_8(n) = 16\sum_{d|n}(-1)^{n+d} d^3$

8.1.11 Number Theory

106. for $i > j$, $\gcd(i, j) = \gcd(i - j, j) \le (i - j)$

107. $\sum_{x=1}^{n}\left[d|x^k\right] = \left\lfloor\frac{n}{\prod_{i=0} p_i^{\left\lceil\frac{e_i}{k}\right\rceil}}\right\rfloor$, where $d = \prod_{i=0} p_i^{e_i}$. Here, $[a|b]$ means if a divides b then it is 1, otherwise it is 0.

108. The number of lattice points on segment (x_1, y_1) to (x_2, y_2) is $\gcd(abs(x_1 - x_2), abs(y_1 - y_2)) + 1$

109. $(n-1)!$ mod $n = n - 1$ if n is prime, 2 if $n = 4$, 0 otherwise.

110. A number has odd number of divisors if it is perfect square

111. The sum of all divisors of a natural number n is odd if and only if $n = 2^r\cdot k^2$ where r is non-negative and k is positive integer.

112. Let a and b be coprime positive integers, and find integers $a\prime$ and $b\prime$ such that $aa\prime \equiv 1$ mod b and $bb\prime \equiv 1$ mod a. Then the number of representations of a positive integers (n) as a non negative linear combination of a and b is

$$\frac{n}{ab} - \left\{\frac{b\prime n}{a}\right\} - \left\{\frac{a\prime n}{b}\right\} + 1$$

Here, x denotes the fractional part of x.

$$\sum_{i=1}^{a}\sum_{j=1}^{b}\sum_{k=1}^{c} d(i\cdot j\cdot k)$$
$$= \sum_{\gcd(i,j)=\gcd(j,k)=\gcd(k,i)=1}\left\lfloor\frac{a}{i}\right\rfloor\left\lfloor\frac{b}{j}\right\rfloor\left\lfloor\frac{c}{k}\right\rfloor$$

Here, $d(x)$ = number of divisors of x.

113. **Gauss's generalization of Wilson's theorem:**, Gauss proved that,

$$\prod_{\substack{k=1\\ \gcd(k,m)=1}}^{m} k$$
$$\equiv \begin{cases} -1 \pmod m & \text{if } m = 4,\ p^\alpha,\ 2p^\alpha \\ 1 \pmod m & \text{otherwise} \end{cases}$$

where p represents an odd prime and α a positive integer. The values of m for which the product is -1 are precisely the ones where there is a primitive root modulo m.

8.1.12 Divisor Function

114. $\sigma_x(n) = \sum_{d|n} d^x$

115. It is multiplicative i.e if $\gcd(a, b) = 1 \to \sigma_x(ab) = \sigma_x(a)\sigma_x(b)$.

116.

$$\sigma_x(n) = \prod_{i=1}^{\tau}\frac{p_i^{(a_i+1)x} - 1}{p_i^x - 1}$$

117. **Divisor Summatory Function**

(a) Let $\sigma_0(k)$ be the number of divisors of k.

(b) $D(x) = \sum_{n\le x}\sigma_0(n)$

(c) $D(x) = \sum_{k=1}^{x}\lfloor\frac{x}{k}\rfloor = 2\sum_{k=1}^{u}\lfloor\frac{x}{k}\rfloor - u^2$, where $u = \sqrt{x}$

(d) $D(n)$ =Number of increasing arithmetic progressions where $n + 1$ is the second or later term. (i.e. The last term, starting term can be any positive integer $\le n$. For example, $D(3) = 5$ and there are 5 such arithmetic progressions: $(1, 2, 3, 4); (2, 3, 4); (1, 4); (2, 4); (3, 4)$.

118. Let $\sigma_1(k)$ be the sum of divisors of k. Then, $\sum_{k=1}^{n}\sigma_1(k) = \sum_{k=1}^{n} k\left\lfloor\frac{n}{k}\right\rfloor$

119. $\prod_{d|n} d = n^{\frac{\sigma_0}{2}}$ if n is not a perfect square, and $= \sqrt{n}\cdot n^{\frac{\sigma_0 - 1}{2}}$ if n is a perfect square.

8.1.13 Euler's Totient function

120. The function is multiplicative. This means that if $\gcd(m,n)=1$, $\phi(m\cdot n)=\phi(m)\cdot\phi(n)$.

121. $\phi(n)=n\prod_{p|n}(1-\frac{1}{p})$

122. If p is prime and $(k\geq 1)$, then: $\phi(p^k)=p^{k-1}(p-1)=p^k(1-\frac{1}{p})$

123. $J_k(n)$, the Jordan totient function, is the number of k-tuples of positive integers all less than or equal to n that form a coprime $(k+1)$-tuple together with n. It is a generalization of Euler's totient, $\phi(n)=J_1(n)$. $J_k(n)=n^k\prod_{p|n}(1-\frac{1}{p^k})$

124. $\sum_{d|n}J_k(d)=n^k$

125. $\sum_{d|n}\phi(d)=n$

126. $\phi(n)=\sum_{d|n}\mu(d)\cdot\frac{n}{d}=n\sum_{d|n}\frac{\mu(d)}{d}$

127. $\phi(n)=\sum_{d|n}d\cdot\mu(\frac{n}{d})$

128. $a|b\to\varphi(a)|\varphi(b)$

129. $n|\varphi(a^n-1)$ for $a,n>1$

130. $\varphi(mn)=\varphi(m)\varphi(n)\cdot\frac{d}{\varphi(d)}$ where $d=gcd(m,n)$ Note the special cases

$$\varphi(2m)=\begin{cases}2\varphi(m) & ;if\ m\ is\ even\\ \varphi(m) & ;if\ m\ is\ odd\end{cases}$$

$$\varphi(n^m)=n^{m-1}\varphi(n)$$

131. $\varphi(lcm(m,n))\cdot\varphi(gcd(m,n))=\varphi(m)\cdot\varphi(n)$ Compare this to the formula $lcm(m,n)\cdot gcd(m,n)=m\cdot n$

132. $\varphi(n)$ is even for $n\geq 3$. Moreover, if if n has r distinct odd prime factors, $2^r|\varphi(n)$

133. $\sum_{d|n}\frac{\mu^2(d)}{\varphi(d)}=\frac{n}{\varphi(n)}$

134. $\sum_{1\leq k\leq n,\gcd(k,n)=1}k=\frac{1}{2}n\varphi(n)$ for $n>1$

135. $\frac{\varphi(n)}{n}=\frac{\varphi(rad(n))}{rad(n)}$ where $rad(n)=\prod_{p|n,p\ prime}p$

136. $\phi(m)\geq\log_2 m$

137. $\phi(\phi(m))\leq\frac{m}{2}$

138. When $x\geq\log_2 m$, then

$$n^x \mod m=n^{\phi(m)+x \mod \phi(m)} \mod m$$

139. $\sum_{1\leq k\leq n,\gcd(k,n)=1}\gcd(k-1,n)=\varphi(n)d(n)$ where $d(n)$ is number of divisors. Same equation for $\gcd(a\cdot k-1,n)$ where a and n are coprime.

140. For every n there is at least one other integer $m\neq n$ such that $\varphi(m)=\varphi(n)$.

141. $\sum_{i=1}^{n}\varphi(i)\cdot\lfloor\frac{n}{i}\rfloor=\frac{n*(n+1)}{2}$

142. $\sum_{i=1,i\%2\neq 0}^{n}\varphi(i)\cdot\lfloor\frac{n}{i}\rfloor=\sum_{k\geq 1}[\frac{n}{2^k}]^2$. Note that $[\,]$ is used here to denote round operator not floor or ceil

143.

$$\sum_{i=1}^{n}\sum_{j=1}^{n}ij[\gcd(i,j)=1]=\sum_{i=1}^{n}\varphi(i)i^2$$

144. Average of coprimes of n which are less than n is $\frac{n}{2}$.

8.1.14 Mobius Function and Inversion

145. It is a multiplicative function.

146.

$$\sum_{d|n}\mu(d)=\begin{cases}1 & ;n=1\\ 0 & ;n>0\end{cases}$$

147. $\sum_{n=1}^{N}\mu^2(n)=\sum_{n=1}^{\sqrt{N}}\mu(k)\cdot\left\lfloor\frac{N}{k^2}\right\rfloor$ This is also the number of square-free numbers $\leq n$

148. **Mobius inversion theorem:** The classic version states that if g and f are arithmetic functions satisfying $g(n)=\sum_{d|n}f(d)$ for every integer $n\geq 1$ then $g(n)=\sum_{d|n}\mu(d)g\left(\frac{n}{d}\right)$ for every integer $n\geq 1$

149. If $F(n)=\prod_{d|n}f(d)$, then $F(n)=\prod_{d|n}F\left(\frac{n}{d}\right)^{\mu(d)}$

150. $\sum_{d|n}\mu(d)\phi(d)=\prod_{j=1}^{K}(2-P_j)$ where p_j is the primes factorization of d

151. If $F(n)$ is multiplicative, $F\not\equiv 0$, then $\sum_{d|n}\mu(d)f(d)=\prod_{i=1}(1-f(P_i))\cdot$ where p_i are primes of n.

8.1.15 GCD and LCM

152. $\gcd(a,0)=a$

153. $\gcd(a,b)=\gcd(b,a \mod b)$

154. Every common divisor of a and b is a divisor of $\gcd(a,b)$.

155. if m is any integer, then $\gcd(a+m\cdot b,b)=\gcd(a,b)$

156. The gcd is a multiplicative function in the following sense: if a_1 and a_2 are relatively prime, then $\gcd(a_1\cdot a_2,b)=\gcd(a_1,b)\cdot\gcd(a_2,b)$.

157. $\gcd(a,b)\cdot\operatorname{lcm}(a,b)=|a\cdot b|$

158. $\gcd(a,\operatorname{lcm}(b,c))=\operatorname{lcm}(\gcd(a,b),\gcd(a,c))$.

159. $\operatorname{lcm}(a,\gcd(b,c))=\gcd(\operatorname{lcm}(a,b),\operatorname{lcm}(a,c))$.

160. For non-negative integers a and b, where a and b are not both zero, $\gcd(n^a-1,n^b-1)=n^{\gcd(a,b)}-1$

161. $\gcd(a,b)=\sum_{k|a\text{ and }k|b}\phi(k)$

162. $\sum_{i=1}^{n}[\gcd(i,n)=k]=\phi\left(\frac{n}{k}\right)$

163. $\sum_{k=1}^{n}\gcd(k,n)=\sum_{d|n}d\cdot\phi\left(\frac{n}{d}\right)$

164. $\sum_{k=1}^{n}x^{\gcd(k,n)}=\sum_{d|n}x^d\cdot\phi\left(\frac{n}{d}\right)$

165. $\sum_{k=1}^{n}\frac{1}{\gcd(k,n)}=\sum_{d|n}\frac{1}{d}\cdot\phi\left(\frac{n}{d}\right)=\frac{1}{n}\sum_{d|n}d\cdot\phi(d)$

166. $\sum_{k=1}^{n}\frac{k}{\gcd(k,n)}=\frac{n}{2}\cdot\sum_{d|n}\frac{1}{d}\cdot\phi\left(\frac{n}{d}\right)=\frac{n}{2}\cdot\frac{1}{n}\cdot\sum_{d|n}d\cdot\phi(d)$

167. $\sum_{k=1}^{n}\frac{n}{\gcd(k,n)}=2*\sum_{k=1}^{n}\frac{k}{\gcd(k,n)}-1$, for $n>1$

168. $\sum_{i=1}^{n}\sum_{j=1}^{n}[\gcd(i,j)=1]=\sum_{d=1}^{n}\mu(d)\lfloor\frac{n}{d}\rfloor^2$

169. $\sum_{i=1}^{n}\sum_{j=1}^{n}\gcd(i,j)=\sum_{d=1}^{n}\phi(d)\lfloor\frac{n}{d}\rfloor^2$

170. $\sum_{i=1}^{n}\sum_{j=1}^{n}i\cdot j[\gcd(i,j)=1]=\sum_{i=1}^{n}\phi(i)i^2$

171. $F(n)=\sum_{i=1}^{n}\sum_{j=1}^{n}\operatorname{lcm}(i,j)=\sum_{l=1}^{n}\left(\frac{(1+\lfloor\frac{n}{l}\rfloor)(\lfloor\frac{n}{l}\rfloor)}{2}\right)^2\sum_{d|l}\mu(d)ld$

172. $\gcd(\operatorname{lcm}(a,b),\operatorname{lcm}(b,c),\operatorname{lcm}(a,c))=\operatorname{lcm}(\gcd(a,b),\gcd(b,c),\gcd(a,c))$

173. $\gcd(A_L,A_{L+1},\ldots,A_R)=\gcd(A_L,A_{L+1}-A_L,\ldots,A_R-A_{R-1})$.

174. Given n, If $SUM=LCM(1,n)+LCM(2,n)+\cdots+LCM(n,n)$ then SUM $=\frac{n}{2}(\sum_{d|n}(\phi(d)\times d)+1$

8.1.16 Legendre Symbol

175. $\left(\frac{a}{p}\right)\equiv a^{\frac{p-1}{2}} \pmod p$ and $\left(\frac{a}{p}\right)\in\{-1,0,1\}$.

176. The Legendre symbol is periodic in its first (or top) argument: if $a\equiv b \pmod p$, then $\left(\frac{a}{p}\right)=\left(\frac{b}{p}\right)$.

177. The Legendre symbol is a completely multiplicative function of its top argument: $\left(\frac{ab}{p}\right)=\left(\frac{a}{p}\right)\left(\frac{b}{p}\right)$

178. The Fibonacci numbers $1,1,2,3,5,8,13\ldots$ are defined by the recurrence $F_1=F_2=1, F_{n+1}=F_n+F_{n-1}$. If p is a prime number then $F_{p-\left(\frac{p}{5}\right)}\equiv 0 \pmod p$, $F_p\equiv\left(\frac{p}{5}\right) \pmod p$.

179. Continuing from previous point, $\left(\frac{p}{5}\right)=$ infinite concatenation of the sequence $(1,-1,-1,1,0)$ from $p\geq 1$.

180. If $n=k^2$ is perfect square then $\left(\frac{n}{p}\right)=1$ for every odd prime except $\left(\frac{n}{k}\right)=0$ if k is an odd prime.

8.1.17 Miscellaneous

181. $a+b=a\oplus b+2(a\&b)$.

182. $a+b=a\mid b+a\&b$

183. $a\oplus b=a\mid b-a\&b$

184. k_{th} bit is set in x iff $x \mod 2^{k-1}\geq 2^k$. It comes handy when you need to look at the bits of the numbers which are pair sums or subset sums etc.

185. k_{th} bit is set in x iff $x \mod 2^{k-1}-x \mod 2^k\neq 0$ $(=2^k$ to be exact). It comes handy when you need to look at the bits of the numbers which are pair sums or subset sums etc.

186. $n \mod 2^i=n\&(2^i-1)$

187. $1\oplus 2\oplus 3\oplus\cdots\oplus(4k-1)=0$ for any $k\geq 0$

188. **Erdos Gallai Theorem:** The degree sequence of an undirected graph is the non-increasing sequence of its vertex degrees A sequence of non-negative integers $d_1\geq d_2\geq\cdots\geq d_n$ can be represented as the degree sequence of finite simple graph on n vertices if and only if $d_1+d_2+\cdots+d_n$ is even and

$$\sum_{i=1}^{k}d_i\leq k(k-1)+\sum_{i=k+1}^{n}\min(d_i,k)$$

holds for every k in $1\leq k\leq n$.

189. $f_n=\sum_{i=0}^{n}\binom{n}{i}g_i\Rightarrow g_n=\sum_{i=0}^{n}(-1)^{n-i}\binom{n}{i}f_i$

8.1.18 Traingle

- **Sine Rule:** $\frac{a}{\sin A}=\frac{b}{\sin B}=\frac{c}{\sin C}=2R$
- **Cosine Rule:** $c^2=a^2+b^2-2ab\cos C$
- **Area:** $\Delta=\sqrt{s(s-a)(s-b)(s-c)}=\frac{1}{2}bc\sin A$
- **Circumradius (R):** $R=\frac{abc}{4\Delta}=\frac{a}{2\sin A}$
- **Inradius (r):** $r=\frac{\Delta}{s}$

- **Ex-radii** (r_a, r_b, r_c)**:** $r_a = \frac{\Delta}{s-a}$
- **Centroid** (G)**:** $\left(\frac{\sum x}{3}, \frac{\sum y}{3}\right)$
- **Incenter** (I)**:** $\left(\frac{ax_1+bx_2+cx_3}{a+b+c}, \frac{ay_1+by_2+cy_3}{a+b+c}\right)$
- **Excenter** (I_A)**:** Replace a with $-a$ in Incenter formula.
- **Euler Line:** O, G, H are collinear; $HG = 2GO$.
- **Median** (m_a)**:** $4m_a^2 = 2b^2 + 2c^2 - a^2$ (Apollonius)
- **Angle Bisector** (l_a)**:** $l_a = \frac{2bc}{b+c}\cos(A/2)$
- **Stewart's Theorem:** Cevian(a line from A to a) length d dividing a into m, n:

$$b^2m + c^2n = a(d^2 + mn)$$

- **Centroid** (G)**:** Intersection of *medians* (vertex to midpoint).
- **Incenter** (I)**:** Intersection of *angle bisectors*.
- **Circumcenter** (O)**:** Intersection of *perpendicular bisectors* of the sides.
- **Orthocenter** (H)**:** Intersection of *altitudes* (vertex perpendicular to opposite side).

8.1.19 Circle

- **General Eq:** $x^2 + y^2 + 2gx + 2fy + c = 0$. Center $(-g, -f)$, $r = \sqrt{g^2 + f^2 - c}$.
- **Tangent at** $P(x_1, y_1)$**:** $xx_1 + yy_1 + g(x + x_1) + f(y + y_1) + c = 0$
- **Slope of tangent from** $P(x, y)$**:** $y + f = m(x + g) \pm r\sqrt{1 + m^2}$
- **Tangency Condition** $(y = mx + c)$**:** $c^2 = r^2(1 + m^2)$
- **Chord Length:** $L = 2\sqrt{r^2 - d^2}$ (d = dist to center)
- **Power of Point** P**:** $d^2 - r^2$ (d = dist P to center) (Useful for determining if a point is inside or outside)
- **Secant Theorem:** $PA \cdot PB = PC \cdot PD$ (AB and CD are 2 lines drawn from point P over the circle)
- **Sector Area:** $\frac{1}{2}r^2\theta$ (θ in rads)
- **Segment Area (Cut by chord):** $\frac{1}{2}r^2(\theta - \sin\theta)$

8.1.20 Points & 2D Line

- **Dist. point** (x_1, y_1) **to line** $ax + by + c = 0$**:**

$$d = \frac{|ax_1 + by_1 + c|}{\sqrt{a^2 + b^2}}$$

- **Dist. between parallel lines:** $d = \frac{|c_1 - c_2|}{\sqrt{a^2+b^2}}$
- **Shoelace Area:** $\frac{1}{2}|\sum(x_iy_{i+1} - x_{i+1}y_i)|$
- **foot of Perpendicular** (h, k):

$$\frac{h - x_1}{a} = \frac{k - y_1}{b} = -\frac{ax_1 + by_1 + c}{a^2 + b^2}$$

- **Reflection** (h, k)**:** Replace $-(\ldots)$ with $-2(\ldots)$ above.
- **Rotation of Axes (CCW** θ**):** $x = X\cos\theta - Y\sin\theta, \quad y = X\sin\theta + Y\cos\theta$

8.1.21 Vector

- **Direction Cosines:** $l = \cos\alpha, m = \cos\beta, n = \cos\gamma$. $l^2 + m^2 + n^2 = 1$.
- **Dot Product:** $\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}|\cos\theta$
- **Projection of a on b:** $(\mathbf{a} \cdot \hat{b})\hat{b}$
- **Cross Product:** Area $\triangle = \frac{1}{2}|\mathbf{a} \times \mathbf{b}|$.
- **Scalar Triple (Vol):** $[\mathbf{abc}] = \mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$. Tet Vol $= \frac{1}{6}|[\ldots]|$.
- **Vector Triple:** $\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = (\mathbf{a} \cdot \mathbf{c})\mathbf{b} - (\mathbf{a} \cdot \mathbf{b})\mathbf{c}$

8.1.22 3D Line

Line 1: $\mathbf{r} = \mathbf{a}_1 + \lambda\mathbf{b}_1$. Line 2: $\mathbf{r} = \mathbf{a}_2 + \mu\mathbf{b}_2$.

- **Angle Between Lines:** $\cos\theta = \frac{|\mathbf{b}_1 \cdot \mathbf{b}_2|}{|\mathbf{b}_1||\mathbf{b}_2|}$
- **Condition for Intersection:** Scalar Triple Product $[\mathbf{a}_2 - \mathbf{a}_1, \mathbf{b}_1, \mathbf{b}_2] = 0$.
- **Intersection Point:** Solve $\mathbf{a}_1 + \lambda\mathbf{b}_1 = \mathbf{a}_2 + \mu\mathbf{b}_2$ for λ, μ.
- **Shortest Dist (Skew):** $D = \left|\frac{(\mathbf{a}_2 - \mathbf{a}_1) \cdot (\mathbf{b}_1 \times \mathbf{b}_2)}{|\mathbf{b}_1 \times \mathbf{b}_2|}\right|$
- **Dist Between Parallel Lines:** $D = \frac{|(\mathbf{a}_2 - \mathbf{a}_1) \times \mathbf{b}|}{|\mathbf{b}|}$
- **Dist Point** P **to Line:** $D = \frac{|(\mathbf{p} - \mathbf{a}) \times \mathbf{b}|}{|\mathbf{b}|}$
- **Foot of Perp** F **from** P**:** $\mathbf{f} = \mathbf{a} + \left(\frac{(\mathbf{p} - \mathbf{a}) \cdot \mathbf{b}}{|\mathbf{b}|^2}\right)\mathbf{b}$
- **Image** P' **in Line:** $\mathbf{p}' = 2\mathbf{f} - \mathbf{p}$.

8.1.23 Plane

Through $\mathbf{a}$, normal $\mathbf{n}$. Eq: $(\mathbf{r} - \mathbf{a}) \cdot \mathbf{n} = 0$.

- **Cartesian:** $Ax + By + Cz + D = 0$. Normal (A, B, C).
- **Dist Point** P **to Plane:** $D = \frac{|Ax_1 + By_1 + Cz_1 + D|}{\sqrt{A^2 + B^2 + C^2}}$
- **Angle Between Planes:** $\cos\theta = \frac{|\mathbf{n}_1 \cdot \mathbf{n}_2|}{|\mathbf{n}_1||\mathbf{n}_2|}$
- **Angle Between Line & Plane:** $\sin\phi = \frac{|\mathbf{b} \cdot \mathbf{n}|}{|\mathbf{b}||\mathbf{n}|}$
- **Intersection (Line of two planes):** Direction $\mathbf{v} = \mathbf{n}_1 \times \mathbf{n}_2$.
- **Family of Planes:** $P_1 + \lambda P_2 = 0$ (Planes through intersection line).
- **Foot of Perp from** P**:** $\frac{x - x_1}{A} = \frac{y - y_1}{B} = \frac{z - z_1}{C} = \frac{-(Ax_1 + By_1 + Cz_1 + D)}{A^2 + B^2 + C^2}$
- **Image of Point in Plane:** Use -2 instead of -1 above.
- **Plane Bisectors:** $\frac{A_1x + B_1y + C_1z + D_1}{\sqrt{A_1^2 + B_1^2 + C_1^2}} = \pm\frac{A_2x + B_2y + C_2z + D_2}{\sqrt{A_2^2 + B_2^2 + C_2^2}}$

8.1.24 Sphere

- **Standard:** $(x - u)^2 + (y - v)^2 + (z - w)^2 = r^2$.
- **Diameter Form:** $(x - x_1)(x - x_2) + (y - y_1)(y - y_2) + (z - z_1)(z - z_2) = 0$.
- **Tangent Plane at** P**:** $(\mathbf{r} - \mathbf{c}) \cdot (\mathbf{p} - \mathbf{c}) = r^2$.
- **Circle of Intersection:** Radius $R = \sqrt{r^2 - d^2}$.
- **Spherical Cosine Rule:** $\cos a = \cos b\cos c + \sin b\sin c\cos A$.

8.1.25 Solid

- **Cone:** $V = \frac{1}{3}\pi r^2 h$, $SA = \pi r(r + l)$
- **Sphere:** $V = \frac{4}{3}\pi r^3$, $SA = 4\pi r^2$
- **Spherical Cap:** $SA = 2\pi rh$. $V = \frac{1}{3}\pi h^2(3r - h)$.
- **Frustum (Cone):** $V = \frac{1}{3}\pi h(R^2 + r^2 + Rr)$. $LSA = \pi(R + r)l$.
- **Pyramid:** $V = \frac{1}{3}(\text{Base Area}) \times h$. $LSA = \frac{1}{2}(\text{Perimeter}) \times l$.
- **Regular Tetrahedron:** Side a. $H = a\sqrt{2/3}$. $V = \frac{a^3}{6\sqrt{2}}$. Face Angle $\cos\theta = 1/3$.

8.1.26 Trigonometry

- $\sin(A \pm B) = \sin A\cos B \pm \cos A\sin B$
- $\cos(A \pm B) = \cos A\cos B \mp \sin A\sin B$
- $\cos 2A = \cos^2 A - \sin^2 A = 2\cos^2 A - 1$
- $\tan^{-1} x \pm \tan^{-1} y = \tan^{-1}\left(\frac{x \pm y}{1 \mp xy}\right)$ *(Note: For +, if $xy > 1$, add π to the result.)*
- $2\tan^{-1} x = \tan^{-1}\left(\frac{2x}{1 - x^2}\right) = \sin^{-1}\left(\frac{2x}{1 + x^2}\right) = \cos^{-1}\left(\frac{1 - x^2}{1 + x^2}\right)$
- $\tan^{-1} x + \cot^{-1} x = \frac{\pi}{2}$